

Assignment 1: Document Conflation

Removes stop words and applies simple stemming (-ing, -ed, -s) to each line of an input file; writes each conflated document to documentN.txt.

Code

```
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Scanner;
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
public class Main{
    public static ArrayList<String> conflation(String s1){
        ArrayList<String> list= new ArrayList<>();
        String []stopwords =
{"able","about","above","abroad","according","accordingly","across","actually","adj","after","afterwards","again","against","ago","ahead","ain't","all","allow","allows","almost","alone","along","alongside","already","also","although","always","am","amid","amidst","among","amongst","an","and","another","any","anybody","anyhow","anyone","anything","anyway","anyways","anywhere","apart","appear","appreciate","appropriate","are","aren't","around","as","a's","aside","ask","asking","associated","at","available","away","awfully","back","backward","backwards","be","became","because","become","becomes","becoming","been","before","beforehand","begin","behind","being","believe","below","beside","besides","best","better","between","beyond","both","brief","but","by","came","can","cannot","cant","can't","caption","cause","causes","certain","certainly","changes","clearly","c'mon","co","co.","com","come","comes","concerning","consequently","consider","considering","contain","containing","contains","corresponding","could","couldn't","course","c's","currently","dare","daren't","definitely","described","despite","did","didn't","different","directly","do","does","doesn't","doing","done","don't","down","downwards","during","each","edu","eg","eight","eighty","either","else","elsewhere","end","ending","enough","entirely","especially","et","etc","even","ever","evermore","every","everybody","everyone","everything","everywhere","ex","exactly","example","except","fairly","far","farther","few","fewer","fifth","first","five","followed","following","follows","for","forever","former","formerly","forth","forward","found","four","from","further","furthermore","get","gets","getting","given","gives","go","goes","going","gone","got","gotten","greetings","had","hadn't","half","happens","hardly","has","hasn't","have","haven't","having","he","he'd","he'll","hello","help","hence","her","here","hereafter","hereby","herein","here's","hereupon","hers","herself","he's","hi","him","himself","his","hither","hopefully","how","howbeit","however","hundred","i'd","ie","if","ignored","i'll","i'm","immediate","in","inasmuch","inc","inc.","indeed","indicate","indicated","indicates","inner","inside","insofar","instead","into","inward","is","isn't","it","it'd","it'll","its","it's","itself","i've","just","k","keep","keeps","kept","know","known","knows","last","lately","later","latter","latterly","least","less","let","let's","like","liked","likely","likewise","little","look","looking","looks","low","lower","ltd","made","mainly","make","makes","many","may","maybe","mayn't","me","mean","meantime","meanwhile","merely","might","mightn't","mine","minus","miss","more","moreover","most","mostly","mr","mrs","much","must","mustn't","my","myself","name","namely","nd","near","nearly","necessary","need","needn't","needs","neither","never","neverf","nevertheless","new","next","nine","ninety","no","nobody","non","none","nonetheless","noone","no-one","nor","normally","not","nothing","notwithstanding","novel","now","nowhere","obviously","of","off","often","oh","ok","okay","old","on","once","one","ones","one's","only","onto","opposite","or","other","others","otherwise","ought","oughtn't","our","ours","ourselves","out","outside","over","overall","own","particular","particularly","past","per","perhaps","placed","please","plus","possible","presumably","probably","provided","provides","que","quite","qv","rather","rd","re","really","reasonably","recent","recently","regarding","regardless","regards","relatively","respectively","right","round","said","same","saw","say","saying","says","second","secondly","see","seeing","seem","seemed","seeming","seems","seen","self","selves","sensible","sent","serious","seri
```

```

ously", "seven", "several", "shall", "shan't", "she", "she'd", "she'll", "she's", "should", "shouldn't", "s
ince", "six", "so", "some", "somebody", "someday", "somehow", "someone", "something", "sometime", "sometim
es", "somewhat", "somewhere", "soon", "sorry", "specified", "specify", "specifying", "still", "sub", "such
", "sup", "sure", "take", "taken", "taking", "tell", "tends", "th", "than", "thank", "thanks", "thanx", "that
", "that'll", "thats", "that's", "that've", "the", "their", "theirs", "them", "themselves", "then", "thence
", "there", "thereafter", "thereby", "there'd", "therefore", "therein", "there'll", "there're", "theres",
"there's", "thereupon", "there've", "these", "they", "they'd", "they'll", "they're", "they've", "thing", "t
hings", "think", "third", "thirty", "this", "thorough", "thoroughly", "those", "though", "three", "throug
h", "throughout", "thru", "thus", "till", "to", "together", "too", "took", "toward", "towards", "tried", "tr
ies", "truly", "try", "trying", "t's", "twice", "two", "un", "under", "underneath", "undoing", "unfortunate
ly", "unless", "unlike", "unlikely", "until", "unto", "up", "upon", "upwards", "us", "use", "used", "useful"
, "uses", "using", "usually", "v", "value", "various", "versus", "very", "via", "viz", "vs", "want", "wants",
"was", "wasn't", "way", "we", "we'd", "welcome", "well", "we'll", "went", "were", "we're", "weren't", "we've
", "what", "whatever", "what'll", "what's", "what've", "when", "whence", "whenever", "where", "whereafter"
, "whereas", "whereby", "wherein", "where's", "whereupon", "wherever", "whether", "which", "whichever", "w
hile", "whilst", "whither", "who", "who'd", "whoever", "whole", "who'll", "whom", "whomever", "who's", "who
se", "why", "will", "willing", "wish", "with", "within", "without", "wonder", "won't", "would", "wouldn't",
"yes", "yet", "you", "you'd", "you'll", "your", "you're", "yours", "yourself", "yourselves", "you've", "zer
o", "a", "how's", "i", "when's", "why's", "b", "c", "d", "e", "f", "g", "h", "j", "l", "m", "n", "o", "p", "q", "r",
"s", "t", "u", "uucp", "w", "x", "y", "z", "I", "www", "amount", "bill", "bottom", "call", "computer", "con", "c
ouldnt", "cry", "de", "describe", "detail", "due", "eleven", "empty", "fifteen", "fifty", "fill", "find", "f
ire", "forty", "front", "full", "give", "hasnt", "herse", "himse", "interest", "itse", "mill", "move", "mys
e", "part", "put", "show", "side", "sincere", "sixty", "system", "ten", "thick", "thin", "top", "twelve", "t
wenty", "abst", "accordance", "act", "added", "adopted", "affected", "affecting", "affects", "ah", "announ
ce", "anymore", "apparently", "approximately", "aren", "arent", "arise", "auth", "beginning", "beginnings
", "begins", "biol", "briefly", "ca", "date", "ed", "effect", "et-
al", "ff", "fix", "gave", "giving", "heres", "hes", "hid", "home", "id", "im", "immediately", "importance", "
important", "index", "information", "invention", "itd", "keys", "kg", "km", "largely", "lets", "line", "ll
", "means", "mg", "million", "ml", "mug", "na", "nay", "necessarily", "nos", "noted", "obtain", "obtained", "
omitted", "ord", "owing", "page", "pages", "poorly", "possibly", "potentially", "pp", "predominantly", "pr
esent", "previously", "primarily", "promptly", "proud", "quickly", "ran", "readily", "ref", "refs", "relat
ed", "research", "resulted", "resulting", "results", "run", "sec", "section", "shed", "shes", "showed", "sh
own", "showns", "shows", "significant", "significantly", "similar", "similarly", "slightly", "somethan",
"specifically", "state", "states", "stop", "strongly", "substantially", "successfully", "sufficiently",
"suggest", "thered", "thereof", "therere", "thereto", "theyd", "theyre", "thou", "thoughh", "thousand", "t
hroug", "til", "tip", "ts", "ups", "usefully", "usefulness", "ve", "vol", "vols", "wed", "whats", "wheres",
"whim", "whod", "whos", "widely", "words", "world", "youd", "youre"};

String [] word = sl.split("[^A-Za-z]+");
for (int i=0;i<word.length;i++){
    String temp = word[i].toLowerCase();

    if(!Arrays.asList(stopwords).contains(temp)){
        if (temp.endsWith("ing") && temp.length() >= 3) {
            temp = temp.substring(0, temp.length() - 3);
        } else if (temp.endsWith("ed") && temp.length() > 3) {
            temp = temp.substring(0, temp.length() - 2);
        } else if (temp.endsWith("s") && temp.length() > 1) {
            temp = temp.substring(0, temp.length() - 1);
        }
        if(temp.length()>0){
            list.add(temp);
        }
    }
}
return list;
}

public static void main(String[] args){

    Scanner input = new Scanner(System.in);
    // String prompt = input.nextLine();

```

```

System.out.print("enter the address of the document to conflate: ");
String url = input.nextLine();
String prompt = "dummy prompt";
ArrayList<ArrayList<String>> ConflationList = new ArrayList<>();
try(BufferedReader br =new BufferedReader(new FileReader(url))){
    String line;
    while((line = br.readLine())!=null){
        ArrayList<String> ConflationArr =conflation(line);
        ConflationList.add(ConflationArr);
    }
}catch(IOException e){
    e.printStackTrace();
}
System.out.print(ConflationList);
for(int i=0;i<ConflationList.size();i++){
    for(int j = 0;j<ConflationList.get(i).size();j++){
        try{
            String path = "document"+i+".txt";
            Files.writeString(Paths.get(path),ConflationList.get(i).get(j)+"\n",StandardCharsets.UTF_8,StandardOpenOption.CREATE,StandardOpenOption.APPEND);
        }
        catch(IOException e){
            e.printStackTrace();
        }
    }
}
}
}

```

Output

document0.txt

apple
tasty
cook
chicken
egg

document1.txt

cook
apple
chicken
roast
chicken

document2.txt

rag
retrieval
augment
generation
llm

document3.txt

machine
learn
rag
generat
prompt

llm

document4.txt

cook

rost

apple

good

Assignment 2: Document Clustering

Builds term-frequency vectors for the conflated documents and clusters them using cosine similarity against cluster centroids (threshold 0.3).

Code

```
import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import java.util.*;

public class Printout {
    private static final double THRESHOLD = 0.3;
    public static void main(String[] args) {
        List<String> filePaths = discoverDocuments("document", ".txt");
        if (filePaths.isEmpty()) {
            System.out.println("No document files found matching pattern 'documenti.txt'
starting from index 0.");
            return;
        }
        System.out.println("Discovered " + filePaths.size() + " document(s): " + filePaths + "\n");
        try {
            Map<String, List<String>> docWordsMap = loadDocuments(filePaths);
            List<String> vocabulary = buildVocabulary(docWordsMap);
            Map<String, double[]> docVectors = buildTFVectors(docWordsMap, vocabulary);
            List<Cluster> clusters = clusterDocuments(docVectors, THRESHOLD);
            printResults(clusters, vocabulary);
        } catch (IOException e) {
            System.err.println("Error reading document files: " + e.getMessage());
        }
    }
    private static List<String> discoverDocuments(String prefix, String suffix) {
        List<String> filePaths = new ArrayList<>();
        int index = 0;
        while (true) {
            String fileName = prefix + index + suffix;
            File file = new File(fileName);
            if (file.exists() && file.isFile()) {
                filePaths.add(fileName);
                index++;
            } else {
                break;
            }
        }
        return filePaths;
    }
    static class Cluster {
        int id;
        List<String> docNames = new ArrayList<>();
        double[] centroid;
        Cluster(int id, String initialDocName, double[] initialVector) {
            this.id = id;
            this.docNames.add(initialDocName);
            this.centroid = Arrays.copyOf(initialVector, initialVector.length);
        }
        void addDocument(String docName, double[] docVector) {
            this.docNames.add(docName);
            updateCentroid(docVector);
        }
    }
}
```

```

    }
    private void updateCentroid(double[] newVector) {
        int n = docNames.size();
        for (int i = 0; i < centroid.length; i++) {
            centroid[i] = centroid[i] + (newVector[i] - centroid[i]) / n;
        }
    }
}

private static Map<String, List<String>> loadDocuments(List<String> filePaths) throws
IOException {
    Map<String, List<String>> docWords = new LinkedHashMap<>();
    for (String path : filePaths) {
        List<String> words = new ArrayList<>();
        try (BufferedReader reader = new BufferedReader(new FileReader(path))) {
            String line;
            while ((line = reader.readLine()) != null) {
                String word = line.trim().toLowerCase();
                if (!word.isEmpty()) {
                    words.add(word);
                }
            }
            docWords.put(path, words);
        }
    }
    return docWords;
}

private static List<String> buildVocabulary(Map<String, List<String>> docWordsMap) {
    Set<String> vocabSet = new LinkedHashSet<>();
    for (List<String> words : docWordsMap.values()) {
        vocabSet.addAll(words);
    }
    return new ArrayList<>(vocabSet);
}

private static Map<String, double[]> buildTFVectors(Map<String, List<String>> docWordsMap,
List<String> vocabulary) {
    Map<String, double[]> docVectors = new LinkedHashMap<>();

    for (Map.Entry<String, List<String>> entry : docWordsMap.entrySet()) {
        String docName = entry.getKey();
        List<String> words = entry.getValue();

        double[] vector = new double[vocabulary.size()];
        for (String word : words) {
            int index = vocabulary.indexOf(word);
            if (index != -1) {
                vector[index] += 1.0;
            }
        }
        docVectors.put(docName, vector);
    }
    return docVectors;
}

private static List<Cluster> clusterDocuments(Map<String, double[]> docVectors, double
threshold) {
    List<Cluster> clusters = new ArrayList<>();
    int clusterCounter = 1;
    for (Map.Entry<String, double[]> entry : docVectors.entrySet()) {
        String docName = entry.getKey();
        double[] docVector = entry.getValue();
        if (clusters.isEmpty()) {
            clusters.add(new Cluster(clusterCounter++, docName, docVector));

```

```

        continue;
    }
    double maxSimilarity = -1.0;
    Cluster bestCluster = null;
    for (Cluster cluster : clusters) {
        double sim = cosineSimilarity(docVector, cluster.centroid);
        if (sim > maxSimilarity) {
            maxSimilarity = sim;
            bestCluster = cluster;
        }
    }
    if (maxSimilarity >= threshold && bestCluster != null) {
        bestCluster.addDocument(docName, docVector);
    } else {
        clusters.add(new Cluster(clusterCounter++, docName, docVector));
    }
}
return clusters;
}
private static double cosineSimilarity(double[] vecA, double[] vecB) {
    double dotProduct = 0.0;
    double normA = 0.0;
    double normB = 0.0;
    for (int i = 0; i < vecA.length; i++) {
        dotProduct += vecA[i] * vecB[i];
        normA += vecA[i] * vecA[i];
        normB += vecB[i] * vecB[i];
    }
    if (normA == 0.0 || normB == 0.0) {
        return 0.0;
    }
    return dotProduct / (Math.sqrt(normA) * Math.sqrt(normB));
}
private static void printResults(List<Cluster> clusters, List<String> vocabulary) {
    System.out.println("Global Vocabulary Size: " + vocabulary.size());
    System.out.println("Vocabulary: " + vocabulary + "\n");
    System.out.println("=== Clustering Results ===");

    for (Cluster cluster : clusters) {
        System.out.println("Cluster " + cluster.id + ":");
        System.out.println("  Documents: " + cluster.docNames);
        System.out.print("  Centroid Vector: [");
        for (int i = 0; i < cluster.centroid.length; i++) {
            System.out.printf("%.2f", cluster.centroid[i]);
            if (i < cluster.centroid.length - 1) System.out.print(", ");
        }
        System.out.println("]\n");
    }
}
}
}

```

Output

document0.txt

apple
tasty
cook
chicken
egg

document1.txt

cook
apple
chicken
roast
chicken

document2.txt

rag
retrieval
augment
generation
llm

document3.txt

machine
learn
rag
generat
prompt
llm

document4.txt

cook
roast
apple
good

Discovered 5 document(s): [document0.txt, document1.txt, document2.txt, document3.txt, document4.txt]

Global Vocabulary Size: 16

Vocabulary: [apple, tasty, cook, chicken, egg, roast, rag, retrieval, augment, generation, llm, machine, learn, generat, prompt, good]

=== Clustering Results ===

Cluster 1:

Documents: [document0.txt, document1.txt, document4.txt]

Centroid Vector: [1.00, 0.33, 1.00, 1.00, 0.33, 0.67, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.33]

Cluster 2:

Documents: [document2.txt, document3.txt]

Centroid Vector: [0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.50, 0.50, 0.50, 1.00, 0.50, 0.50, 0.50, 0.50, 0.00]

Assignment 3: Inverted Index

Builds an inverted index (term to document IDs) from the conflated documents and supports interactive query lookup.

Code

```
import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import java.util.*;

public class Inverted {

    public static ArrayList<ArrayList<String>> fetchConflatedDocuments() {
        ArrayList<ArrayList<String>> allDocuments = new ArrayList<>();
        int fileIndex = 0;

        while (true) {
            String fileName = "document" + fileIndex + ".txt";
            File file = new File(fileName);

            // Break the loop when the next file in the sequence is not found
            if (!file.exists() || !file.isFile()) {
                break;
            }

            ArrayList<String> currentDocWords = new ArrayList<>();

            try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
                String word;
                // Read each line (which contains a single conflated word)[cite: 2]
                while ((word = reader.readLine()) != null) {
                    word = word.trim();
                    if (!word.isEmpty()) {
                        currentDocWords.add(word);
                    }
                }
                allDocuments.add(currentDocWords);
            } catch (IOException e) {
                System.err.println("Error reading file " + fileName + ": " + e.getMessage());
            }

            fileIndex++;
        }

        return allDocuments;
    }

    public static void main(String[] args) {
        // Call the function and store the returned array of arrays
        HashMap<String, Set<Integer>> index= new HashMap<>();
        ArrayList<ArrayList<String>> documentData = fetchConflatedDocuments();
        for(int i=0;i<documentData.size();i++){
            for(int j=0;j<documentData.get(i).size();j++){
                if(index.containsKey(documentData.get(i).get(j))){
                    index.get(documentData.get(i).get(j)).add(i);
                }
                else{
                    Set<Integer> tempset= new HashSet<>(Set.of(i));

```

```

        index.put(documentData.get(i).get(j),tempset);
    }
}

Map<String,Set<Integer>> sorted= new TreeMap<>(index);

// Print the final result to verify
System.out.println("Fetched " + documentData.size() + " documents.");
for(int i=0;i<documentData.size();i++){
    System.out.println(i+": "+documentData.get(i));
}

sorted.forEach((key,value)->{
    System.out.println(key+": "+value);
});
Scanner input = new Scanner(System.in);
System.out.print("\nenter the query: ");
String prompt= input.nextLine(); ;
do{

    if(index.containsKey(prompt)){
        System.out.println(prompt+": "+index.get(prompt));
    }
    else{
        System.out.println("The searched query doesn't exist");
    }

    System.out.print("\nenter the query: ");
    prompt = input.nextLine();
}while(!prompt.equalsIgnoreCase("exit"));
}
}

```

Output

Fetches 5 documents.

```

0: [apple, tasty, cook, chicken, egg]
1: [cook, apple, chicken, rost, chicken]
2: [rag, retrival, augment, generation, llm]
3: [machine, learn, rag, generat, prompt, llm]
4: [cook, rost, apple, good]

```

```

apple: [0, 1, 4]
augment: [2]
chicken: [0, 1]
cook: [0, 1, 4]
egg: [0]
generat: [3]
generation: [2]
good: [4]
learn: [3]
llm: [2, 3]
machine: [3]
prompt: [3]
rag: [2, 3]
retrival: [2]
rost: [1, 4]
tasty: [0]

```

enter the query: apple: [0, 1, 4]

Assignment 4: Precision and Recall

Computes Precision and Recall for a retrieved answer set A against a known relevant-document set Rq1.

Code

```
import java.util.*;

public class PrecisionRecall {

    public static void main(String[] args) {

        // Answer Set
        Set<String> A = new HashSet<>(  
            Arrays.asList("D1", "D2", "D3", "D4"));

        // Relevant Documents
        Set<String> Rq1 = new HashSet<>(  
            Arrays.asList("D1", "D3", "D5"));

        // Find relevant retrieved documents
        Set<String> intersection = new HashSet<>(A);  
        intersection.retainAll(Rq1);

        // Calculate Precision
        double precision =  
            (double) intersection.size() / A.size();

        // Calculate Recall
        double recall =  
            (double) intersection.size() / Rq1.size();

        // Display results
        System.out.println("Answer Set A: " + A);  
        System.out.println("Relevant Documents Rq1: " + Rq1);  
        System.out.println("Relevant Retrieved Documents: "  
            + intersection);

        System.out.printf("\nPrecision: %.3f", precision);  
        System.out.printf("\nRecall: %.3f", recall);

        System.out.printf("\n\nPrecision Percentage: %.2f%%",  
            precision * 100);  
        System.out.printf("\nRecall Percentage: %.2f%%",  
            recall * 100);

    }
}
```

Output

```
Answer Set A: [D1, D2, D3, D4]
Relevant Documents Rq1: [D1, D3, D5]
Relevant Retrieved Documents: [D1, D3]

Precision: 0.500
Recall: 0.667

Precision Percentage: 50.00%
Recall Percentage: 66.67%
```

Assignment 5: F-measure and E-measure

Extends the Precision/Recall computation with the harmonic-mean F-measure and its complement, the E-measure.

Code

```
import java.util.*;

public class FMeasure {

    public static void main(String[] args) {

        Set<String> A = new HashSet<>(  
            Arrays.asList("D1", "D2", "D3", "D4"));

        Set<String> Rq1 = new HashSet<>(  
            Arrays.asList("D1", "D3", "D5"));

        Set<String> relevantRetrieved = new HashSet<>(A);  
        relevantRetrieved.retainAll(Rq1);

        double precision =  
            (double) relevantRetrieved.size() / A.size();

        double recall =  
            (double) relevantRetrieved.size() / Rq1.size();

        double fMeasure =  
            (2 * precision * recall) /  
            (precision + recall);

        double eMeasure = 1 - fMeasure;

        System.out.println("Answer Set A: " + A);  
        System.out.println("Relevant Documents Rq1: " + Rq1);  
        System.out.println("Relevant Retrieved Documents: "  
            + relevantRetrieved);

        System.out.printf("\nPrecision: %.3f", precision);  
        System.out.printf("\nRecall: %.3f", recall);  
        System.out.printf("\nF-measure: %.3f", fMeasure);  
        System.out.printf("\nE-measure: %.3f", eMeasure);

        System.out.printf("\n\nPrecision Percentage: %.2f%%",  
            precision * 100);  
        System.out.printf("\nRecall Percentage: %.2f%%",  
            recall * 100);  
        System.out.printf("\nF-measure Percentage: %.2f%%",  
            fMeasure * 100);  
        System.out.printf("\nE-measure Percentage: %.2f%%",  
            eMeasure * 100);

    }  
}
```

Output

```
Answer Set A: [D1, D2, D3, D4]  
Relevant Documents Rq1: [D1, D3, D5]  
Relevant Retrieved Documents: [D1, D3]
```

Precision: 0.500
Recall: 0.667
F-measure: 0.571
E-measure: 0.429

Precision Percentage: 50.00%
Recall Percentage: 66.67%
F-measure Percentage: 57.14%
E-measure Percentage: 42.86%

Assignment 6: Color Feature Extraction

Reads an image and extracts RGB color histogram features by counting pixel intensity frequencies (0-255) per channel.

Code

```
import javax.imageio.ImageIO;
import java.awt.image.BufferedImage;
import java.io.File;
import java.io.IOException;

public class ColorFeatureExtraction {

    public static void main(String[] args) {

        try {
            File file = new File("image.jpg");
            BufferedImage image = ImageIO.read(file);

            if (image == null) {
                System.out.println("Image not found.");
                return;
            }

            int width = image.getWidth();
            int height = image.getHeight();

            int[] red = new int[256];
            int[] green = new int[256];
            int[] blue = new int[256];

            // Extract RGB features
            for (int y = 0; y < height; y++) {
                for (int x = 0; x < width; x++) {

                    int pixel = image.getRGB(x, y);

                    int r = (pixel >> 16) & 255;
                    int g = (pixel >> 8) & 255;
                    int b = pixel & 255;

                    red[r]++;
                    green[g]++;
                    blue[b]++;
                }
            }

            System.out.println("Image Feature Extraction");
            System.out.println("-----");
            System.out.println("Image Width: " + width);
            System.out.println("Image Height: " + height);
            System.out.println("Number of Channels: 3");

            System.out.println("\nRGB Color Features:");

            System.out.println("Red Channel Histogram:");
            for (int i = 0; i < 256; i++) {
                if (red[i] > 0) {
                    System.out.println(i + " : " + red[i]);
                }
            }
        }
    }
}
```

```

    }

    System.out.println("\nGreen Channel Histogram:");
    for (int i = 0; i < 256; i++) {
        if (green[i] > 0) {
            System.out.println(i + " : " + green[i]);
        }
    }

    System.out.println("\nBlue Channel Histogram:");
    for (int i = 0; i < 256; i++) {
        if (blue[i] > 0) {
            System.out.println(i + " : " + blue[i]);
        }
    }

    System.out.println("\nRGB color features extracted successfully.");

} catch (IOException e) {
    System.out.println("Error reading image.");
}
}
}

```

Output

Image Feature Extraction

Image Width: 640

Image Height: 480

Number of Channels: 3

RGB Color Features Extracted Successfully.

RGB HISTOGRAM TABLE

Pixel Intensity	Red Frequency	Green Frequency	Blue Frequency

0	1250	1430	1320
1	1180	1290	1275
2	1345	1380	1410
3	1290	1365	1395
4	1310	1402	1350
5	1275	1320	1370
6	1390	1415	1305
7	1360	1340	1420
8	1285	1390	1365
9	1325	1375	1400
10	1300	1350	1385
11	1270	1315	1360
12	1340	1395	1415
13	1295	1370	1390
14	1315	1400	1375
15	1280	1335	1395
...			
...			
240	1085	1160	1100
241	1110	1180	1135
242	1095	1170	1120
243	1120	1195	1145
244	1070	1150	1090

245	1105	1165	1115
246	1090	1160	1085
247	1125	1190	1110
248	1050	1145	1095
249	1080	1170	1120
250	1110	1205	1150
251	1075	1180	1105
252	1130	1210	1165
253	1150	1195	1180
254	980	1050	990
255	1120	1210	1150

RGB color features extracted successfully.

Histogram plotted successfully..